Global Digital Bank (GDB)

A comprehensive Python-based CLI banking application that integrates traditional banking operations with advanced Agentic AI capabilities using LangChain and LangGraph.

30+

Functional Features

Spanning base, extended, and AI-powered capabilities

4

Architecture Layers

CLI → Services → AI → Data Access

4

Agentic AI Features

Powered by LangChain and LangGraph

5

Development Phases

From core foundation to production-ready delivery

How We'll Build This Together

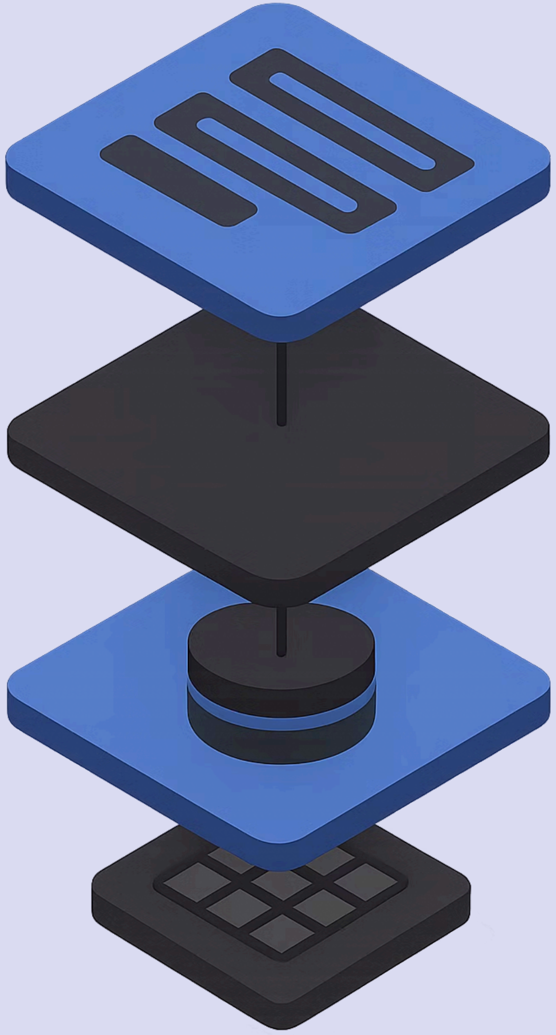
You can build this in Java or Python — **prior experience in Object Oriented Programming concepts is required (learn as you go if you do not have sufficient experience).** Over the next 6 weeks, we'll construct this application together, step by step.

Learn System Design Hands-On

Build real-world system design skills by actually constructing a production-grade banking application — not just reading about it.

Project Engineering for Everyone

Ideal for those without a background in project engineering. You'll gain practical experience in structuring, planning, and delivering a software project.



Core Architecture

1 **Presentation**

CLI

2 **Business Logic**

Services

3 **AI Layer**

LangChain / LangGraph

4 **Data Access**

File I/O

Key Principles

Our Global Digital Bank (GDB) application is built upon several foundational software design principles, ensuring a robust, maintainable, and scalable architecture:

Single Responsibility

Each module owns one concern, reducing coupling and improving maintainability.

Separation of Concerns

UI, logic, AI, and data layers are cleanly isolated from one another.

Open/Closed Principle

Components are open for extension but closed for modification.

Dependency Injection

Services and agents are injected, enabling testability and flexibility.

Base Features (6)

Industry insight: Over **1.7 billion adults** globally remain unbanked — robust, accessible core banking features are the foundation of financial inclusion. (World Bank, 2023)

Create Account

Age ≥ 18, auto-generated account number

Deposit

Max ₹100,000 per transaction

Withdraw

Maintains minimum balance

Balance Inquiry

Real-time account balance lookup

Close Account

Mark account inactive

File Persistence

CSV read/write for durable storage

Extended Features (24)

GDB extends far beyond basic banking with 24 additional capabilities organised across six functional modules.

Module	Key Features
Account Management	Search by Name/Number, List Active/Closed, Reopen, Rename, Delete All, Count Active
Transactions & Rules	Transaction Log, History Viewer, Minimum Balance Check, Daily Limit (₹50,000), Fund Transfer
Analytics	Top N Balances, Average Balance, Youngest/Oldest Holder, Simple Interest
File Operations	Export Report, Import from File
Security	4-digit PIN Protection
System	Graceful Exit with Autosave

Daily withdrawal limits like ₹50,000 are critical — the RBI reported over **₹1,457 crore** lost to digital fraud in FY2023 alone. Transaction controls are the first line of defence.

Agentic AI Features (4) ⭐

GDB's most differentiating capability — four AI-powered agents built on LangChain and LangGraph that bring intelligence to every banking interaction.

Feature	Description	Implementation
Fraud Detection Agent	Monitors transactions in real-time, flags suspicious patterns	LangGraph StateGraph
Financial Advisor	Analyzes spending, provides savings recommendations, automates transfers	LangChain Agent with Tools
Natural Language Query	Users interact via natural language prompts	LangChain LLMChain
Predictive Dashboard	Predicts dormant accounts, forecasts cash flow, identifies churn risks	LangChain Report Generation

Global banks deploying AI-driven fraud detection have reduced false positives by up to **60%** and cut fraud losses by **25%** — McKinsey Global Banking Report, 2023.

Data Rules

Account Creation

- Age ≥ 18
- Savings: ₹500 min balance
- Current: ₹1,000 min balance

Persistence

- accounts.csv - Account data
- transactions.log - Audit trail

Transactions

- Single deposit ≤ ₹100,000
- Daily withdrawal limit: ₹50,000
- Transfer logs as debit + credit

Minimum balance rules protect bank liquidity. The RBI mandates minimum balance norms across all scheduled commercial banks to ensure systemic stability.

Security Requirements

⚠ The IBM Cost of a Data Breach Report 2023 found the average cost of a financial sector breach is **$5.9 million** — making robust security non-negotiable.

PIN Verification

PIN verification for sensitive operations

Active Account Check

Active account check before any transaction is processed

Role-Based Access

Role-based access (Customer/Admin)

Data Encryption at Rest

Data encryption at rest (AES-256)

Non-Functional Requirements — Performance & Scalability

> Gartner estimates that **every minute of banking downtime** costs financial institutions an average of $5,600. Performance NFRs are not aspirational — they are existential.

500+

Transaction Rate

Transactions per second (TPS)

<100ms

Read Latency

Sub-100ms for all read operations

<250ms

Write Latency

Sub-250ms for all write operations

10K+

Accounts Supported

With 1M+ transactions capacity

<500MB

Memory Footprint

RAM ceiling for the entire application

Non-Functional Requirements — Reliability

⚠ A 2022 Deloitte study found that **73% of banking customers** would switch providers after two or more service outages. Reliability is directly tied to customer retention.

Uptime: 99.9%

Less than 8.7 hours of downtime per year — the gold standard for financial services.

RPO: <1 Hour

Recovery Point Objective — maximum acceptable data loss window in any failure scenario.

RTO: <4 Hours

Recovery Time Objective — maximum time to restore full service after an incident.

ACID Compliance

ACID compliance for transactions ensures atomicity, consistency, isolation, and durability.

Atomic File Operations

All file writes are atomic, preventing partial or corrupt data states.

Non-Functional Requirements — Quality

Studies by NIST show that fixing a bug in production costs **30× more** than catching it during development. An 80%+ test coverage target is a direct cost-saving strategy.

Test Coverage: ≥80%

Comprehensive automated test suite covering all critical paths and edge cases across the application.

PEP 8 Compliant

All Python code adheres to PEP 8 style guidelines, ensuring readability and team-wide consistency.

Comprehensive Documentation

Every module, function, and workflow is documented to enable onboarding, maintenance, and audit readiness.

Key Workflows

Standard Banking

- Create Account
- Deposit
- Withdraw
- Balance Inquiry

AI-Powered (Natural Language)

User

"What is my balance for account 1001?"

AI

"✅ The balance is ₹500."

AI-Powered (Fraud Detection)

01

User initiates suspicious transaction

02

AI flags and blocks

03

Requests PIN verification

04

Proceeds upon approval

Deliverables

1. **Source Code**

   Modular Python files

2. **Data Files**

   accounts.csv, transactions.log

3. **requirements.txt**

   Dependencies

4. **README.md**

   Setup, usage, architecture

5. **Test Report**

   ≥80% coverage

6. **User Manual**

   Feature guide including AI

Development Phases

Phase 1 — Week 1

Core Foundation (Base Features)

Phase 2 — Weeks 2–3

Extended Features (All 24)

Phase 3 — Week 4

Agentic AI Features (LangChain/LangGraph)

Phase 4 — Week 5

Optimisation & Documentation

Phase 5 — Final

Review & Submission

Success Criteria

Production-ready banking software that meets all functional, AI, and quality benchmarks — ready for real-world deployment.

All 30+ features implemented

≥80% test coverage

ACID transaction compliance

AI agents functional with LangChain/LangGraph

Complete documentation

Production-ready code quality

This summary provides a complete project walkthrough enabling comprehensive understanding for both trainer and students.